

1. a. Greedy algoritmalar local olarak her adımda en iyi sonucu bulmak, global olarakta en iyisini bulmayı hedefler.

→ Greedy algoritmalar optimizasyon problemleri için kullanılırlar.

↳ Her adımda bir seçim yaparak globalde-(Tüm problem) optimal sonucu bulmak hedeflenir.

b. Greedy Algoritmalar her zaman globalde optimal sonucu vermezler.

Amaç ora adımlar ile localde iyi olanı bulmaktır.

→ Örneğin change making te ilk gelen ilk cıkarı yada ilk biteni hemen almak localde iyi iken globalde optimal değildir.

3. Fractional Knapsack greedy çözülebilir ama 0-1 Knapsack çözülmez. (\*)  
Optimal solution elde edilemez.

Örnek 1

|                 |                   |                |
|-----------------|-------------------|----------------|
| $w = 30$<br>(5) | $x = \frac{p}{s}$ | $\frac{s}{10}$ |
|                 | $y = 10$          | 10             |
|                 | $z = 1$           | 20             |
|                 | $k = 9$           | 9              |

Price en düşük al

$\frac{z+x}{(1)(5)}$  kapasiteyi doldurur.

(6)  $\rightarrow$  2 eleman seçer

Ama

$\frac{x+y+k}{(5)(10)(9)}$  şeklinde daha iyi seçim yapılabilir.

$\Rightarrow 24$   $\rightarrow$  3 eleman seçer

Price göre sıralı  
içinde w'ler var.

min PriceKnap(arr[0..n], w)

count = 0

for i in range [0..n-1]

if w == 0 break

if arr[i] < w

w = w - arr[i]

++ count

return count

Worst  $\rightarrow O(n)$

Best  $\rightarrow O(1)$

$w = 0$ .

Average  $\rightarrow O(n)$

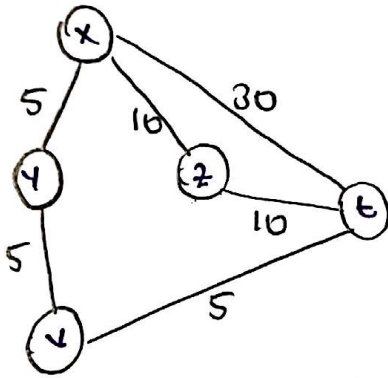
w 0 değil ama uygun eleman yok.

4. Bu problem prim algoritması ile çözülebilir. Kruskal'da çözüm sağlanacaktır.

↓  
Cycle kontrolü yapılmalı

### Algoritma

- Edge al
- En yakın komşu edge seç
- Sıradaki için aynı işi uygula



x'ten t'ye

$x \rightarrow y \rightarrow v \rightarrow t$

\* Her zaman doğru sonuç olmaz.

|   | x            | y            | z  | v | t  |
|---|--------------|--------------|----|---|----|
| x | 0            | 5            | 10 | 0 | 30 |
| y | <del>5</del> | 0            | 0  | 5 | 0  |
| z | 10           | 0            | 0  | 0 | 10 |
| v | 0            | <del>5</del> | 0  | 0 | 5  |
| t | 30           | 0            | 10 | 5 | 0  |

$$5+5+5 = 15$$

5.

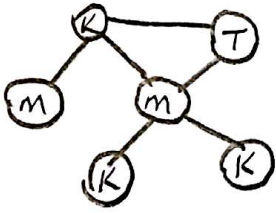
Algorithm

→ İlk vertexi ilk renk ile boyar

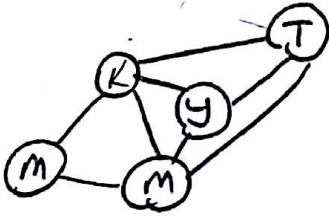
→ Geri kalan vertexler için:

→ Sıradaki vertexi komşularından farklı olacak şekilde boyar. (Boya komşularda olmayan renkten olmalı.)

Bi-partite graph gibidir ama en az 2 renk ile tamamen çözülmez.



# En az 4 renk gereklidir.  
← 3 renkli örnek



Graph in bağlantı komşusluğu arttıkça kullanılması gereken minimum renk sayısı artar.

NP-Complete bir problemdir.

6. Her işletim sisteminin kendine özgü algoritmaları var.

Kullanılan algoritmalarda bir tane ilk gelen ilk çıkar yöntemidir.

| <u>Instruction</u> | <u>Time</u> | <u>Coming time</u> |
|--------------------|-------------|--------------------|
| X                  | 5           | 0                  |
| Y                  | 10          | 1                  |
| Z                  | 5           | 2                  |

X ilk önce işlenir. X bitmeden Y ve Z gelir.

Sıraya göre aldığı için  $Y \rightarrow Z$  olarak alır.

Z kısa sürede bitecek olmasına rağmen Y seçilir.

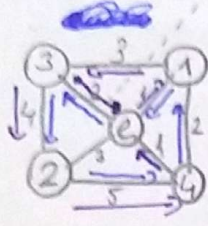
Bu da optimal ~~sol~~ değildir.

$X \rightarrow Z \rightarrow Y$  olarak alması daha doğru olur.

Z 10t beklemek yerine Y'yi 5t bekletir.



2. a.



→ Mor olan yolun ağırlığı

$1 \xrightarrow{2} e \xrightarrow{3} 3 \xrightarrow{4} 2 \xrightarrow{5} 4$

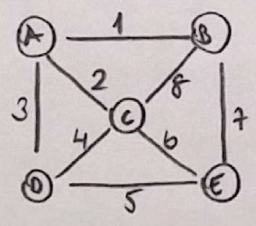
$3 \xrightarrow{2} e \xrightarrow{1} 1 \xrightarrow{2} 4 \xrightarrow{5} 2$

→ e her 2 minimum spanning tree içinde olmalı

TRUE

b. Bu sorunun cevabı için yukarıdaki yol (MST) örnekleri ele alınırsa,  $4 \rightarrow e \rightarrow 1 \rightarrow 3 \rightarrow 2$  yolu eklendiğinde ortak olmayan yollar vardır. Buda false olur.

c.

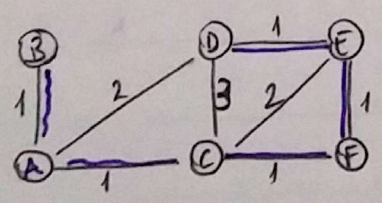


→  $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$

Weightlar farklı olduğu sürece min. 1 MST vardır.

TRUE

d.



FALSE

Burada aynı tree'ler ortaya çıkar

⇒ Başlangıç noktaları aynı olanlarda

D-E-F-C-A-B

7. a,

→ ilk sıtirden son sıtira kadar her sıtiran en küçük sıtımını seç

→ Seçilen sıtım ve sıtım sil

→ Son sıtırda kalan sıtımın sonucu olarak al

b, Algoritma her zaman optimal solution üretmez.

min. al

|              |              |              |
|--------------|--------------|--------------|
| <del>6</del> | <del>3</del> | <del>2</del> |
| 5            | 1            | 4            |
| <del>3</del> | 7            | 8            |

$$2+1+3=6$$

17 > 6

Optimal Sonucu vermedi.

Disprove

|          |   |   |
|----------|---|---|
| <u>6</u> | 3 | 2 |
| 5        | 1 | 4 |
| 3        | 7 | 8 |

$$6+4+7=\underline{\underline{17}}$$

8.

procedure change\_making(arr[1...n], x)

for  $i \leftarrow 1$  to  $n$

changes[i]  $\leftarrow \lfloor n / \text{arr}[i] \rfloor$

$m \leftarrow m \bmod \text{arr}[i]$

if  $n == 0$  return changes

else return ""

Time Efficiency  $O(n)$



9. a) Prim algoritmasında spanning tree bulabilmek için tüm vertexler bağlı ve aralarında minimum bir weight olmak zorunda. Eğer bize ağırlıkları olmayan graph verilirse, kendimiz tüm mesafelere eşit bir sayı verip çözebiliriz. Örneğin tüm weight'ler 5 olsun.

b. Her zaman en küçük weight'e sahip edge gitmek işlem sonunda minimum spanning tree yi veremeyebilir.  
Breadth First Search ile daha düzgün sonuçlar elde edilebilir.